

AI Engineer Assessment Brief

RAG Chatbot · 3–4 hours

See the chatbot you need to build

Try the live AI assistant demo — it uses the 10 company PDFs and Claude. Build a similar experience using RAG with Python.

[Open Chatbot](#)

Use Case

SWS AI has 10 internal company policy documents (HR, leave, resignation, IT security, etc.). Employees currently have to open each PDF manually to find answers. You will build a **RAG (Retrieval Augmented Generation) chatbot** that lets employees ask natural language questions like *"How many sick leaves do I get?"* and receive accurate, grounded answers sourced directly from the company documents — without hallucination.

Important Rules — Read Before Starting

- › **Language: Python.** Backend must be in Python. Use any framework (FastAPI recommended).
- › **RAG approach only.** Do not fine-tune a model. Use retrieval + generation with the provided documents.
- › **Use any vector database** — Chroma is recommended for simplicity, but Pinecone, FAISS, Qdrant all work.
- › **Any LLM is fine** — OpenAI, Anthropic Claude, Gemini, or a local model via Ollama.
- › **Be ready to explain your approach** — chunking strategy, embedding model choice, retrieval k value, and prompt design.
- › **Show sources:** Your chat UI must display which documents were used to generate each answer.



Assessment Duration: 3–4 hours

Steps 1–4 are all required. Focus on getting the RAG pipeline working first.



Commit every 15 min

RAG Architecture Overview



What to Build

Follow these steps in order. Each step builds on the previous one.

01 Document Ingestion Pipeline Required

Load and process the 10 company PDF documents into a format suitable for retrieval.

- ✓ Load the 10 SWS AI company PDF documents (available for download below).
- ✓ Parse and extract text from the PDFs using a library like PyMuPDF, pdfplumber, or LangChain's document loaders.
- ✓ Split the text into chunks using a text splitter (e.g., RecursiveCharacterTextSplitter with chunk_size=500, chunk_overlap=50).
- ✓ Generate embeddings for each chunk using an embedding model (OpenAI text-embedding-3-small, HuggingFace sentence-transformers, or similar).
- ✓ Store the embeddings and their associated metadata (document name, chunk index, page number) in a vector database.

02 Vector Database Setup Required

Store document embeddings in a vector store for fast semantic retrieval.

- ✓ Choose any vector database: Chroma (easiest, local), Pinecone, Weaviate, Qdrant, FAISS, or pgvector.
- ✓ Create a collection/index for the SWS AI documents.
- ✓ Store each chunk with its embedding vector, the source document name, and the raw text.
- ✓ Test that you can retrieve the top-k most relevant chunks for a sample query like 'What is the leave policy?'
- ✓ Document your vector DB choice and why in the README.

03

**RAG-Powered Chatbot API** Required

Build a Python backend that accepts a user question, retrieves relevant document chunks, and generates a grounded answer using an LLM.

- ✓ Build a Python API (FastAPI or Flask) with a POST /api/chat endpoint that accepts { question: string }.
- ✓ On each request: (1) embed the question, (2) retrieve the top-3 to top-5 most relevant chunks from the vector store, (3) pass the chunks + question to an LLM as context.
- ✓ The LLM should be instructed to answer only from the provided context. If the answer is not in the documents, respond: 'I don't have that information in the company documents.'
- ✓ Use any LLM: OpenAI GPT-4o/GPT-3.5, Anthropic Claude, Google Gemini, or a local model like Ollama.
- ✓ Use LangChain, LlamaIndex, or raw API calls — any framework is fine.
- ✓ Include the source document names in the API response alongside the answer.

04

**Chat UI** Required

Build a simple chat interface that connects to your RAG backend. This is what you will demo.

- ✓ Build a web-based chat UI (React, Vue, plain HTML — your choice).
- ✓ The UI should have a message input field, a send button, and a conversation thread showing user messages and AI responses.
- ✓ Show a loading/typing indicator while the backend is processing the query.
- ✓ Display the source documents used to generate the answer (e.g., 'Sources: HR Policy, Leave Policy').
- ✓ The 'Start Demo' button on the assessment portal should lead directly to this chat interface.
- ✓ Try to match the white and blue design language of this page. Font: Livvic.

</> Framework Options**LangChain** Python

Most popular framework. Great for chains, agents, and document loaders.

LlamaIndex Python

Excellent for document ingestion and retrieval workflows.

Raw API Python

Use OpenAI/Anthropic/Gemini SDK directly if you prefer more control.

Vector Database Options

Chroma Recommended

Local, easy setup, perfect for prototypes. pip install chromadb.

FAISS

Facebook AI Similarity Search — in-memory, fast, no server needed.

Pinecone

Managed cloud vector DB — free tier available.

Qdrant

Open-source, can run locally with Docker.

Sample Queries to Test Your Chatbot

Your chatbot should be able to answer all of these from the provided documents.



"What is the annual leave policy at SWS AI?"



"How many days of sick leave do employees get?"



"What is the notice period for resignation?"



"What tools does SWS AI use for communication?"



"What is the password policy for company systems?"



"How are performance reviews conducted?"



"What are the WFH guidelines?"



"Does SWS AI offer health insurance?"

What to Submit

- ✓ A public GitHub repository with clear commit history (commit at least every 15 minutes).
- ✓ A README with setup steps: how to ingest documents, set environment variables, and run the API and chat UI.
- ✓ A brief explanation of your architecture decisions — embedding model, vector DB, chunking strategy, retrieval k, prompt design.
- ✓ Optionally, a screen recording demonstrating the chatbot answering sample queries and showing the source documents — bonus, not required.

Company Documents

Download all 10 company policy documents before starting your assessment. These PDFs are what your application will work with.

✓ All Downloaded

Important: Download these PDFs and keep them handy. Your application must allow uploading these documents and should reference them for the chatbot / RAG feature (AI track).



Company Overview & Mission

About SWS AI



HR Policy

Human Resources



Leave Policy

Human Resources



Resignation & Exit Policy

Human Resources



Code of Conduct

Workplace Standards



Work From Home Policy

Workplace Standards



Performance Review Policy

Career Development



Benefits & Compensation

Career Development



Employee Onboarding Guide

Getting Started



IT & Security Policy

Compliance



